GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Sesión 05 — Diseño de arquitectura y diagramas técnicos

2º CFGS DAW/DAM · Proyecto Intermodular II

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra

🗺️ La sesión en un mapa

El arte de **diseñar y comunicar** la arquitectura de un sistema mediante diagramas: desde la visión de alto nivel hasta los detalles técnicos que servirán de **contrato entre equipos**.

🏗️
Arquitectura
Monolito vs. microservicios

🔭
Modelo C4
Contexto y contenedores

🗄️
ERD
Modelo de base de datos

🎭
Casos de uso
El QUÉ del sistema

⏱️
Secuencia
El CÓMO interactúan

✍️
DaC
Mermaid y diagramas como código

ℹ️ Ejemplo central: el ecosistema completo de **TaskFlow**, con perspectivas específicas para DAW, DAM y ASIR.

🏛️ ¿Qué es la arquitectura de software?

> *El conjunto de **decisiones estructurales fundamentales**: componentes, responsabilidades y cómo se comunican entre sí.*

Es el **plano maestro** que permite que equipos distintos (DAW, DAM, ASIR) construyan piezas que encajan a la primera.

La arquitectura...	Para qué
Define la estructura	Qué piezas existen y de qué son responsables
Fija la comunicación	APIs, mensajes, colas: los contratos entre equipos
Guía el despliegue	Dónde vive cada componente (nube, red, servidores)
Documenta	Punto de partida de todos los diagramas técnicos

⚠️ Una decisión arquitectónica mal tomada se nota en cada sprint posterior: primero el plano, después el ladrillo.

🔀 Estilos arquitectónicos: monolito vs. microservicios

🧱 Monolito

Todo el código en una sola aplicación, base de datos única, despliegue como unidad.

+ Simplicidad inicial, desarrollo rápido
− Difícil de escalar, alto acoplamiento

🧩 Microservicios

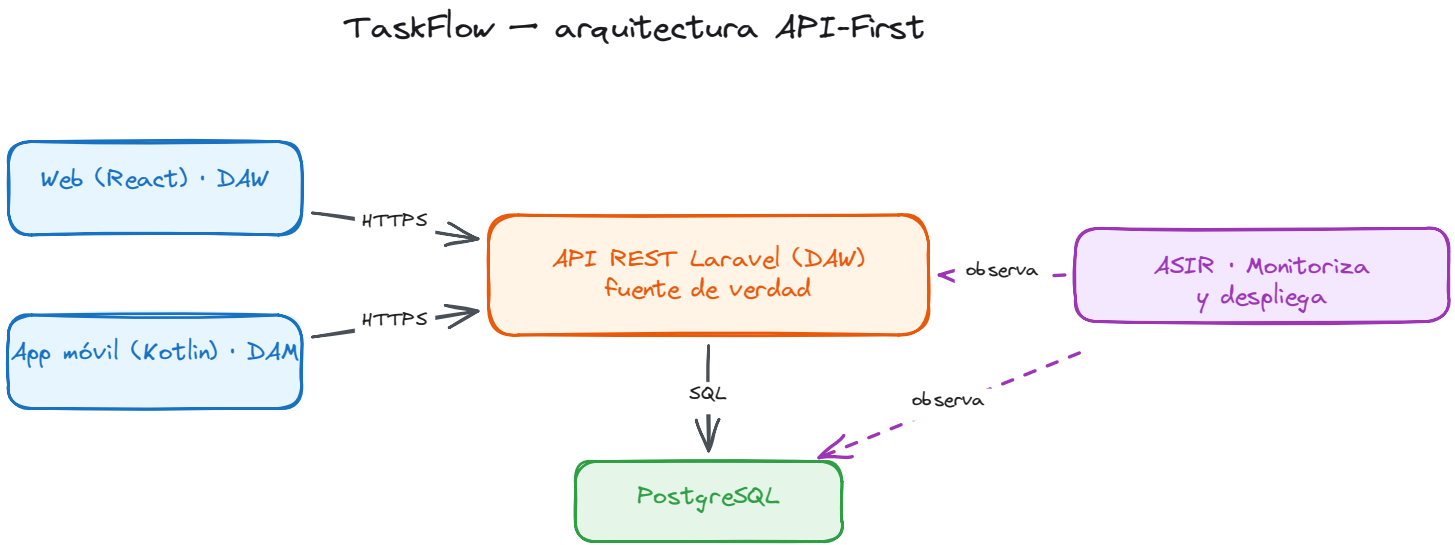
Servicios pequeños e independientes, cada uno con su base de datos, comunicación por APIs.

+ Escalabilidad, resiliencia, equipos autónomos
− Complejidad operacional, latencia de red

ℹ️ Encaja el estilo con el equipo y el calendario: TaskFlow empieza con una sola API para ganar velocidad sin cerrar la puerta a escalar.

🌐 Nuestro enfoque: API-First (híbrido)

La API es la **fuente de verdad**: antes de escribir frontend o móvil, se define el contrato HTTP.



ℹ️ Un solo backend (Laravel), dos clientes (web y app) y una infraestructura que lo observa todo: quedaros con esta imagen, todos los diagramas derivan de ella.

🏗️ Arquitectura API-First: vistas por equipo

La misma decisión estructural, tres lecturas distintas:

Perfil	Su punto de vista	Su diagrama clave
DAW	La API expone endpoints REST a todos los clientes	Contenedores + secuencia
DAM	La app móvil es un cliente más, con capacidad *offline*	Casos de uso + secuencia
ASIR	El sistema completo debe ser observable, escalable y resiliente	Despliegue + red

1. Backend (DAW) expone la API REST como fuente de verdad, documentada y versionada.
2. Web (DAW) y app móvil (DAM) consumen la **misma API**: no se duplica lógica.
3. Infraestructura (ASIR) soporta, despliega y monitoriza el conjunto.

> ⚠️ El contrato API se negocia **antes** de que los tres equipos empiecen a programar: después, cada cambio rompe el trabajo de los demás.

📝 ¿Qué es un contrato API?

Un contrato API describe **qué peticiones acepta** la API y **qué responde**: rutas, métodos, parámetros y cuerpo de la respuesta.

```
Ejemplo: crear una tarea en TaskFlow

  POST /api/tareas HTTP/1.1
  Authorization: Bearer <token>
  Content-Type: application/json

  { "titulo": "Escribir informe", "proyecto_id": 3 }


  HTTP/1.1 201 Created
  { "id": 42, "titulo": "Escribir informe", "estado": "pendiente" }
```

ℹ️ Documentado con OpenAPI/Swagger, este contrato es la referencia de los tres equipos: DAW lo implementa, DAM y la web lo consumen, ASIR lo monitoriza.

🔭 El Modelo C4: zoom progresivo

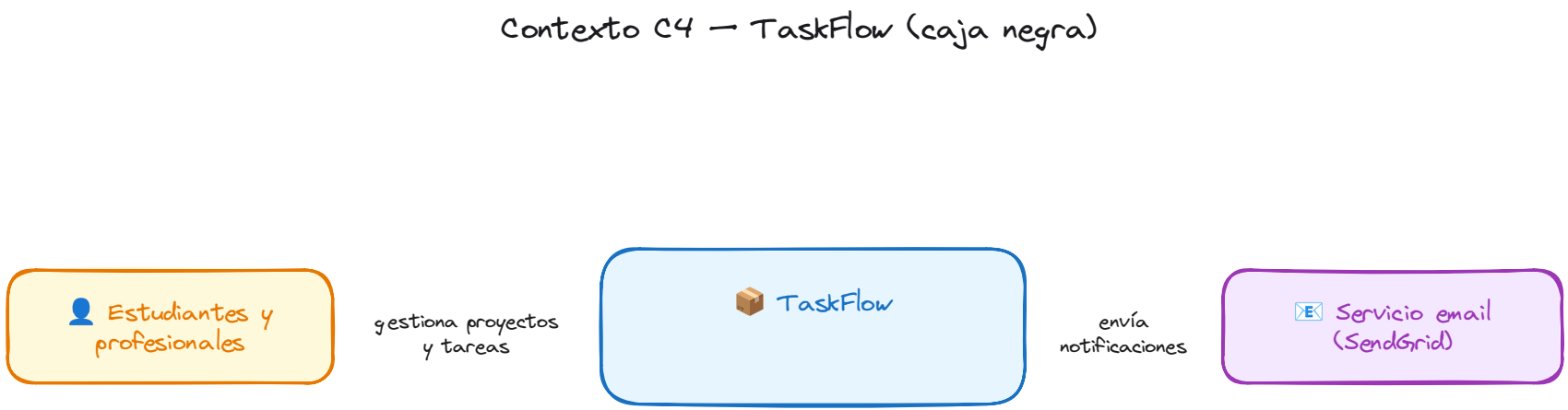
C4 = Context, Containers, Components, Code: abstracciones para visualizar la arquitectura en distintos niveles de zoom.

Nivel	Qué muestra	Para quién
1. Contexto	El sistema como caja negra y su entorno	Todo el mundo (dirección, cliente)
2. Contenedores	Los bloques técnicos: web, app, API, BD, colas	Los equipos técnicos
3. Componentes	Módulos internos de cada contenedor	El equipo que lo construye
4. Código	Clases y relaciones (a menudo, código fuente)	El desarrollador

ℹ️ Hoy trabajamos los niveles 1 y 2, y el diagrama de despliegue: son los que piden los proyectos intermodulares. Como el zoom de un mapa: país → ciudad → calle.

🔭 Nivel 1 — Diagrama de contexto

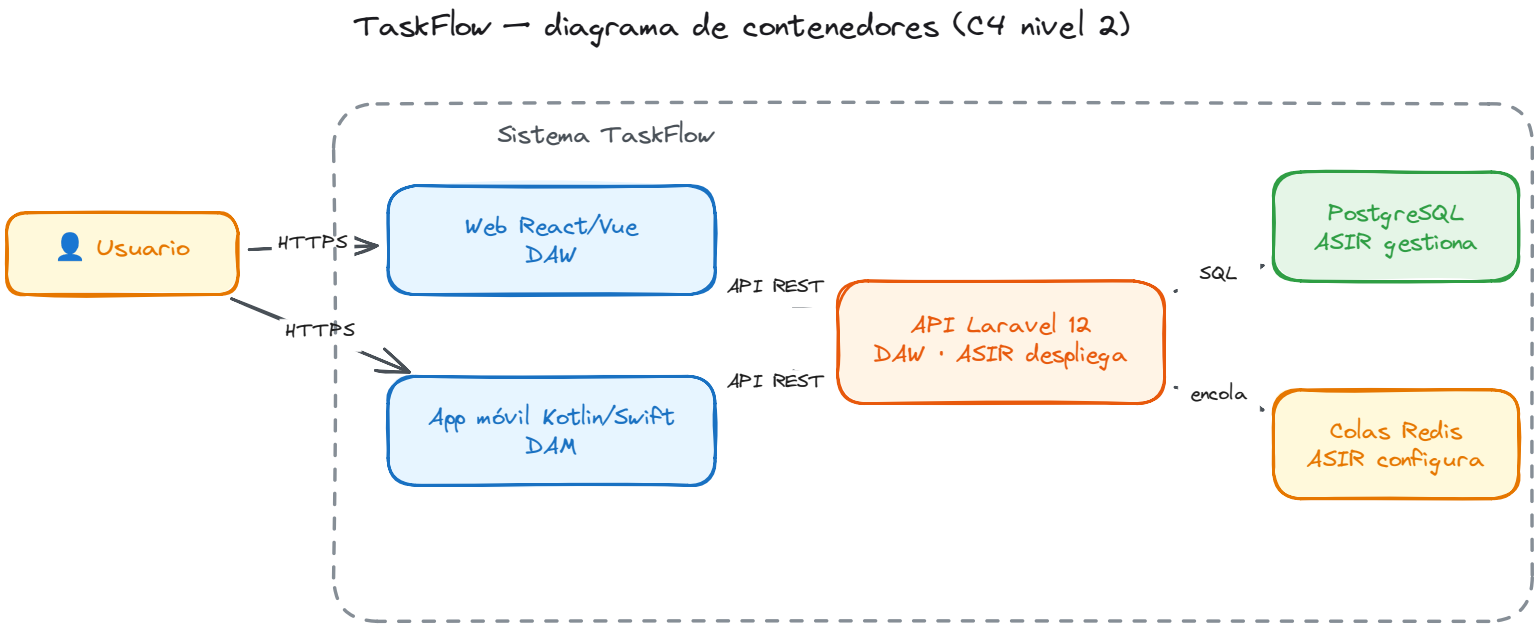
El sistema como **caja negra**: qué entra y qué sale, sin detalles internos.



ℹ️ Cuatro o cinco líneas bastan: si un *stakeholder* no entiende tu contexto en 30 segundos, sobra detalle.

📦 Nivel 2 — Diagrama de contenedores

Los **bloques técnicos principales** y las peticiones que fluyen entre ellos.



ℹ️ Cada contenedor señala quién lo trabaja: DAW desarrolla la web y la API; DAM la app móvil; ASIR gestiona Postgres, Redis y despliegue.

🚀 Diagrama de despliegue

Dónde vive cada componente: servidores, nube, redes, subnets… Responsabilidad principal: **ASIR**, aunque los tres equipos deben entenderlo.

El flujo (TaskFlow)

1. **git push** a GitLab → pipeline CI/CD (tests + build).
2. El **Load Balancer** reparte HTTPS:443: web en subnet pública, API en subnet privada.
3. La API persiste en **RDS PostgreSQL** Multi-AZ y sube archivos a **S3**.

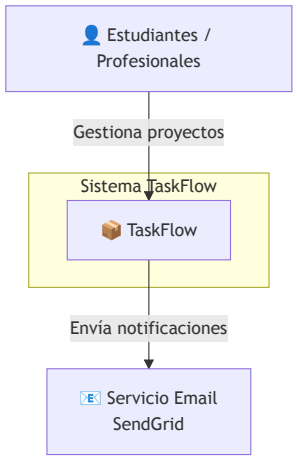
Lecturas por equipo

- **DAW**: dónde corre Laravel (EC2) y por dónde entra el tráfico.
- **DAM**: distribución por tiendas + *Firebase Cloud Messaging* para push.
- **ASIR**: subnets, security groups, Multi-AZ, monitorización con CloudWatch.

⚠️ **No confundir contenedores (lógico) con despliegue (físico): un EC2 puede alojar varios contenedores; la subnet pública expone la web, la privada resguarda API y BD.**

⌨️ Diagrama de contexto en Mermaid

El mismo diagrama de contexto, escrito como texto: se versiona con Git.



Lectura por equipo (resumen): DAW → la API expone endpoints REST · DAM → cliente más con capacidad *offline* · ASIR → observable, escalable y resiliente.

> ℹ️ Lo importante no es el dibujo: es la **idea** que comunica.

🗄️ Diagramas Entidad-Relación (ERD)

El **plano de la base de datos**: tablas, columnas, tipos y relaciones. Se diseña **antes** de escribir una línea de SQL. Responsabilidad principal: **DAW** (pero DAM y ASIR deben entenderlo).

Componentes clave:

Componente	Descripción	Ejemplo
Entidades	Conceptos del negocio	USUARIOS, PROYECTOS, TAREAS
Atributos	Propiedades de cada entidad	id, nombre, email, password_hash
Relaciones	Cardinalidad entre tablas	1:1 perfil, 1:N proyecto→tareas, N:M usuarios↔proyectos (tabla pivote)

Reglas de oro

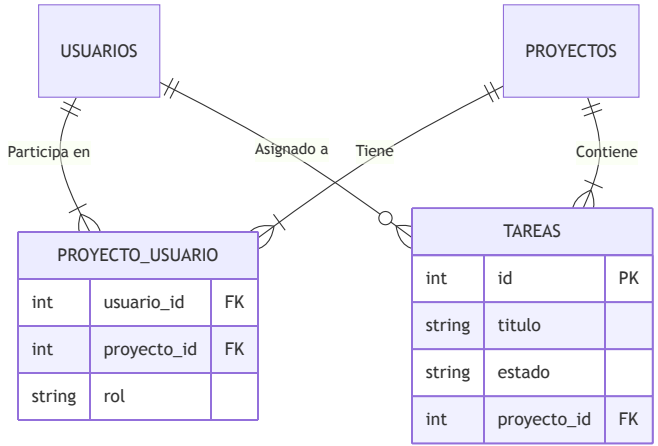
- Toda tabla lleva **id** (PK) autoincremental; las ajenas, `<entidad>_id` (FK).
- Emails y valores únicos llevan **UK**; datos sensibles, siempre hasheados.

Cómo se lee el N:M

- Usuarios ↔ Proyectos = **tabla pivote**: `PROYECTO_USUARIO (usuario_id FK, proyecto_id FK, rol)`.
- El atributo `rol` vive en la relación, no en las entidades.

ℹ️ El N:M no existe en SQL: siempre se materializa con una tabla pivote. Si no la ves en el diagrama, falta.

🗄️ ERD de TaskFlow (básico)



Consultas que facilita el diseño:

```sql
-- Tareas pendientes del usuario 7 con su proyecto
SELECT t.titulo, p.nombre, t.estado
FROM tareas t
JOIN proyectos p ON p.id = t.proyecto_id
WHERE t.usuario_asignado_id = 7 AND t.estado <> 'completada';
```

ℹ️ Un buen ERD responde preguntas de negocio con una sola consulta: si te cuesta escribir el SQL, el modelo está mal.

🎭 Diagramas de casos de uso

Describen **QUÉ** hace el sistema desde la perspectiva de un actor. Propósito: documentar los requisitos funcionales de forma visual. Responsabilidad principal: **DAM** (móvil) y **DAW** (web).

Componentes

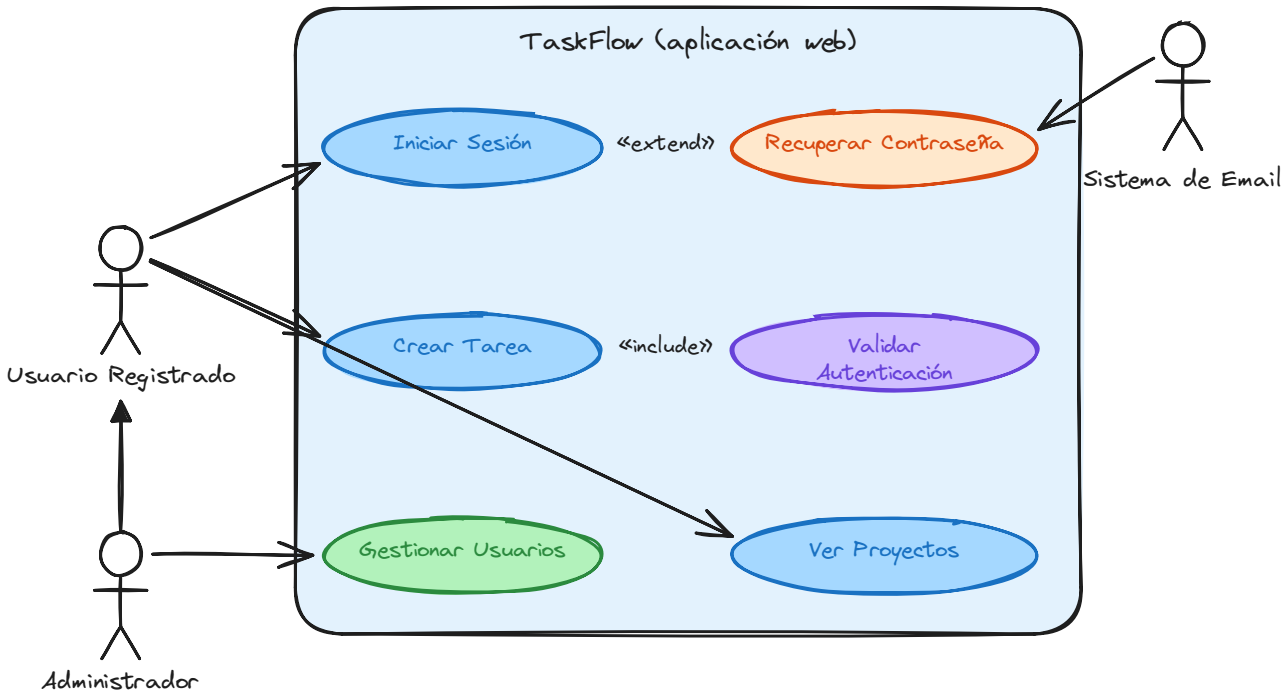
- **Actores**: Usuario Registrado, Administrador, Sistema de Email.
- **Casos de uso**: Iniciar Sesión, Crear Tarea, Ver Proyectos.
- **Herencia**: un Administrador hereda todo lo del Usuario Registrado.

Relaciones

- **«include»** (obligatoria): todas las acciones incluyen *Validar Autenticación*.
- **«extend»** (opcional): *Crear tarea* puede extenderse con *Adjuntar archivo*.
- **Regla práctica**: ¿siempre ocurre? «include». ¿Solo a veces? «extend».

ℹ️ Ejemplo TaskFlow web, en la siguiente diapositiva: «Crear Tarea» incluye «Validar Autenticación» (siempre), «Recuperar Contraseña» extiende a «Iniciar Sesión» (solo a veces) y el Administrador hereda del Usuario Registrado.

🎭 Ejemplo: casos de uso de TaskFlow (web)



ℹ️ Lectura: continuas = actor–caso de uso · «include» (*siempre*): *Crear Tarea* → *Validar Autenticación* · «extend» (*opcional*): *Recuperar Contraseña* → *Iniciar Sesión* · triángulo: el *Administrador* hereda del *Usuario* (y añade *Gestionar Usuarios*).

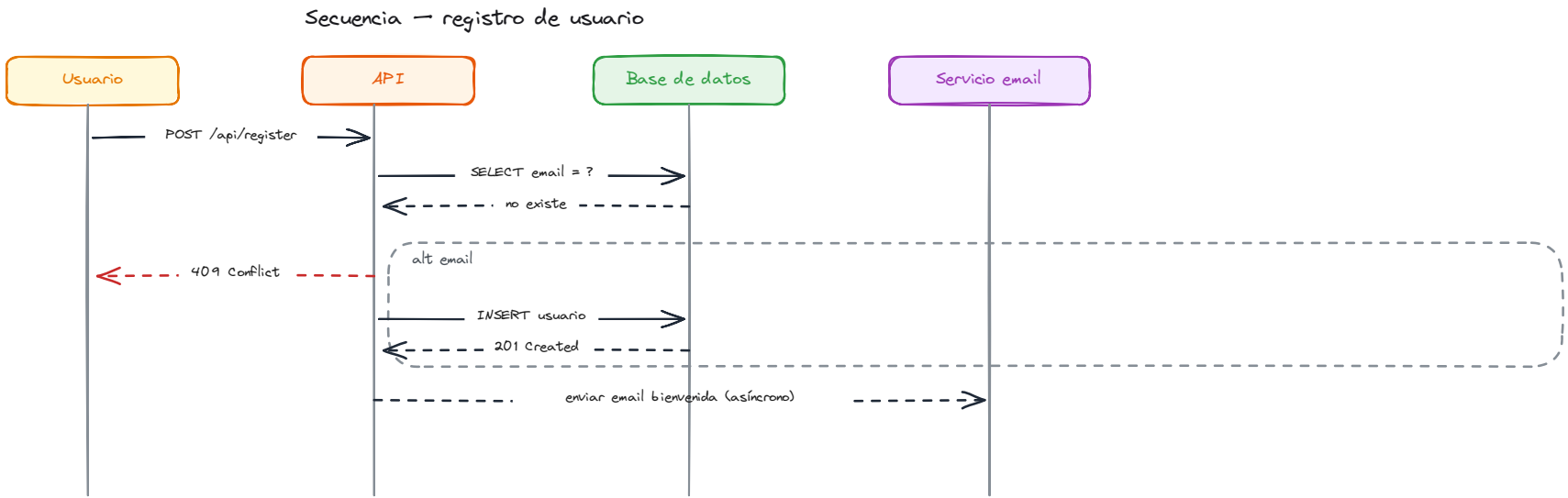
⏱️ Diagramas de secuencia

Muestran **CÓMO** interactúan los componentes a lo largo del tiempo, en un flujo concreto. Todos los equipos diseñan sus flujos.

Elemento	Significado	Notación
Participantes	Componentes que interactúan	Líneas de vida verticales
Mensaje síncrono	El emisor espera respuesta	`->>` flecha sólida
Mensaje asíncrono	El emisor no espera respuesta	`-->>` flecha discontinua
alt / opt	Alternativa u opcional (if-else)	Bloque con etiqueta
loop / par	Bucle u operaciones paralelas	Bloque con etiqueta

ℹ️ Regla de estilo: cada mensaje lleva nombre de operación y datos; si no cabe, el flujo es demasiado grande y hay que dividirlo.

⏱️ Secuencia: registro de usuario



ℹ️ Fijaos en el fragmento «alt»: email duplicado → 409 · email nuevo → INSERT + 201. El correo de bienvenida va con flecha discontinua: asíncrono, la API no espera.

🛠️ Herramientas visuales (WYSIWYG)

Lo que ves es lo que tocas: dibujas con el ratón y el diagrama queda tal cual.

Draw.io (diagrams.net)

- Gratuita y open source.
- Extensión en VS Code, integración con Google Drive y GitHub.
- Bibliotecas de iconos: AWS, Azure, UML, redes.
- **Ideal para**: despliegue y red (ASIR), prototipos rápidos de arquitectura.

Lucidchart

- Interfaz profesional, colaboración en tiempo real.
- Comentarios, revisiones y plantillas.
- **Contras**: plan gratuito muy limitado (3 diagramas).

ℹ️ Son la vía rápida para ilustraciones puntuales; su problema: el diagrama vive fuera de Git y es difícil de versionar y comparar.

✍️ Diagramas como código (Mermaid / DaC)

En lugar de dibujar, **describes el diagrama escribiendo texto**; la herramienta lo genera. Vive en el repo, versionado con el código.

✅ DaC: sí

Se versiona en **Git**: cambia con el código, en el mismo commit.
Fuente única de verdad: nada de PNGs perdidos en Drive.
Renderizado nativo en GitHub, GitLab, VS Code, Notion.

⚠️ DaC: cuidado

Sintaxis que aprender (aunque es sencilla).
Layouts complejos (cloud, VLANs) mandan mejor en Draw.io.
Híbrido: Mermaid + herramienta visual = lo mejor de ambos.

Sintaxis básica (flowchart): `A[Inicio] ⟶ B{¿Autenticado?} → Sí: Dashboard / No: Login`

ℹ️ Con **flowchart**, **erDiagram** y **sequenceDiagram** cubrís el 90 % del proyecto.

🧭 ¿Qué diagrama uso cada vez?

Guía rápida para elegir la herramienta correcta en cada situación del proyecto:

Pregunta que quiero responder	Diagrama	Autor principal
¿Con quién se relaciona el sistema y qué le ofrecen terceros?	Contexto C4	Los tres
¿Qué bloques técnicos forman TaskFlow y quién los construye?	Contenedores C4	DAW / DAM
¿Dónde corre cada cosa (nube, redes, subnets)?	Despliegue	ASIR
¿Cómo se organizan los datos?	ERD	DAW
¿Qué puede hacer cada actor?	Casos de uso	DAW / DAM
¿Qué mensajes se pasan y en qué orden?	Secuencia	Los tres
¿Qué heredan los componentes de infraestructura?	Despliegue + red	ASIR

ℹ️ Antes de dibujar, pregúntate a **quién** le va a servir: el público manda sobre la herramienta.

✍️ DaC aplicado: de la idea al diagrama

De idea a diagrama en tres pasos:

1. Escribe el **flujo en texto** (participantes y mensajes): es el borrador más rápido.
2. Tradúcelo a **Mermaid** y déjalo en `docs/diagramas.md`, versionado con el código.
3. Si el layout se hace inmanejable, **migra a Draw.io** y deja en el .md un enlace.

Pseudocódigo del flujo de registro (lo que alimentará el diagrama de secuencia):

```
SI no existe email:
  hashear contraseña (bcrypt)
  INSERT usuario
  devolver 201 Created
  enviar email bienvenida (asíncrono)
SI NO:
  devolver 409 Conflict
```

> ℹ️ Escribir el pseudocódigo antes del diagrama evita olvidar el camino de error: el 90 % de los flujos reales son «todo va bien» … hasta que no.

✅ Resumen de la sesión

- La **arquitectura de software** son las decisiones estructurales que permiten que tres equipos construyan piezas que encajan: en TaskFlow, estilo **API-First** con la API como fuente de verdad.
- El **modelo C4** da zoom progresivo: contexto (caja negra) → contenedores (bloques técnicos) → despliegue (dónde vive).
- El **ERD** es el plano de la BD; las N:M se materializan con tabla pivote.
- Los **casos de uso** documentan el *qué* (include/extend); la **secuencia**, el *cómo* (alt, síncrono/asíncrono).
- **DaC con Mermaid**: diagramas versionados junto al código; WYSIWYG (Draw.io, Lucidchart) para layouts complejos.
- Los **ejemplos** de cada diagrama (contexto, ERD, casos de uso, secuencia) sirven de plantilla para los del proyecto.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //